Darren Lee
CSE13S

Week 6 Notes

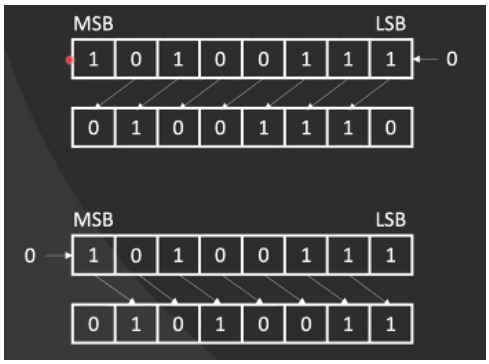
5/3/21
Bit Vectors and Sets

Units of information in a computer

Unit	Size in Bits	Value	Notes
Bit	1	0/1	Smallest
Nibble	4	Hex digit	
Byte	8	ASCII	Smallest addressable
Half word	16		
Word	32		Native size, register length
Long Word	64		Native size, register length

Byte is smallest addressable unit, every byte has a unique address

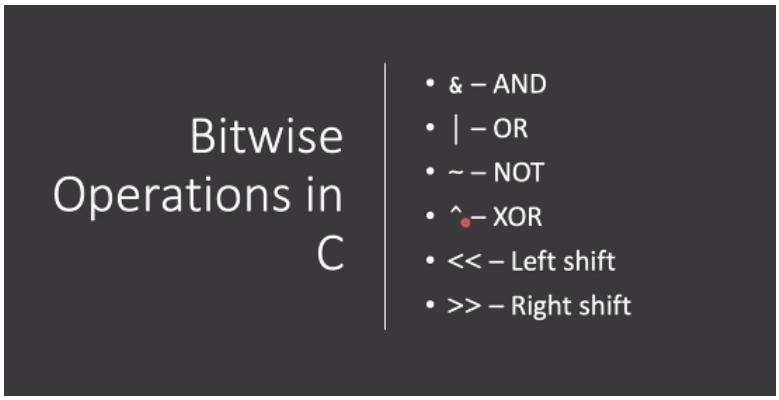
Logical shift
- Logical shift left: zeroes are shifted in on the right
- Logical shift right: zeroes are shifted in on the left



Arithmetic shift
- Arithmetic shift left: zeroes are shifted in on the right
- Arithmetic shift right: sign bits are shifted in on the left

-

Can't count on division or multiplication by 2 with arithmetic shift



C's Vexatious Right Shift (>>)

- The result of v1 >> v2 is v1 right-shifted v2 bits
- If v1 is unsigned, or is signed with a non-negative value
    - v1>> v2 == integral v2 part of the quotient v1/2^(v2)
- If v1 is signed with a negative value
    - v1>> v2 == implementation defined
        - We don't know for sure

C's Vexatious Left Shift <<

- The result of v1<<v2 is v1 left-shifted v2 bits
    - Zeroes are filled
- If v1 is unsigned
    - v1<<v2==v1*2^(v2) modulo n
        - N is maximum value of the resulting type+1
- If v1 is signed with a non-negative value
    - We don't know bc it's implementation defined

AND (&) Truth Table

&	0	1
0	0	0
1	0	1

OR (|) Truth Table

|	0	1
0	0	1
1	1	1

XOR (^) Truth Table

^	0	1
0	0	1
1	1	0

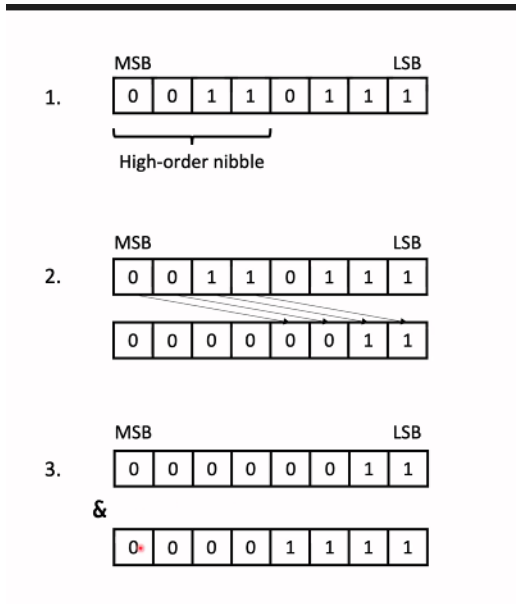
NOT (~) Truth Table

~	
0	1
1	0

Getting a High-Order Nibble

1. A high-order nibble in a byte means the most significant 4 bits
2. Bit-shift right 4 times so that the high-order nibble takes the place of the low-order nibble

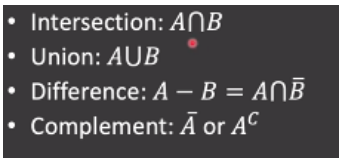
3. AND with 0x0F



Setting a high-order nibble

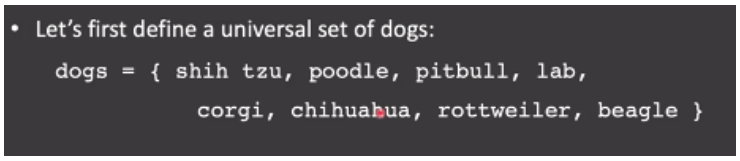
1. We want to place a nibble into the higher-order bits of a byte
2. AND byte with 0x0F
3. Bit-shift nibble left 4 times
4. OR byte with bit-shifted nibble

Sets

- Well-defined unordered collections that are characterized by the elements they contain
- Sets are equivalent if and only if they have exactly the same elements
- Basic relation in set theory is membership
- Operations:
  - 
    - Intersection: $A \cap B$
    - Union: $A \cup B$
    - Difference: $A - B = A \cap \bar{B}$
    - Complement: $\bar{A}$ or A^C

Set operations with dogs

dogs={ }


- Let's first define a universal set of dogs:

```
dogs = { shih tzu, poodle, pitbull, lab,
         corgi, chihuahua, rottweiler, beagle }
```

Set Intersection (A ∩ B)

- The set of elements in A and B.
- `good_dogs = { pitbull, lab, corgi, beagle }`
- `bad_dogs = { shih tzu, poodle, pitbull }`
- `good_dogs ∩ bad_dogs = { pitbull }`

Set intersection is AND

Set Union (A ∪ B)

- The set of elements in A or B.
- `overrated_dogs = { shih tzu, poodle }`
- `cute_dogs = { corgi, beagle }`
- `overrated_dogs ∪ cute_dogs = { shih tzu, poodle, corgi, beagle}`

Set union is OR

Set Difference (A – B)

- The set of elements in A that aren't in B.
- `some_dogs = { shih tzu, poodle, corgi, beagle }`
- `cute_dogs = { corgi, beagle }`
- `some_dogs - cute_dogs = { shih tzu, poodle }`

Set difference is AND NOT

Set Complement ($\bar{A}$ or A^C)

- The set of elements not in A.
- Also defined as U – A, where U is the universal set.
- `dogs = { shih tzu, poodle, pitbull, lab, corgi, chihuahua, rottweiler, beagle }`
- `cute_dogs = { corgi, beagle }`
- `cute_dogs`^{C} `= { shih tzu, poodle, pitbull, lab, chihuahua, rottweiler }`

Set complement is NOT

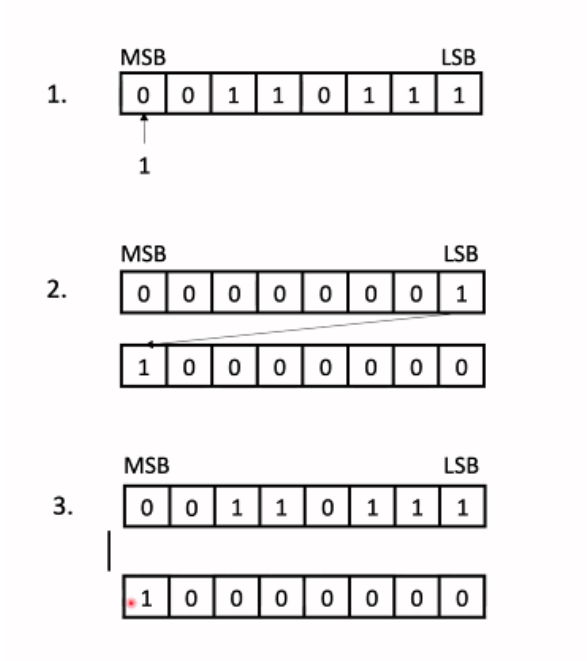
`bits =` { 1 , 0 , 0 , 1 , 1 , 0 , 1 , 1 }

Representing Sets with Bits

- Sets can be represented with bits.
- A 0 indicates that the element is not a member of the set.
- A 1 indicates that the element is a member of the set.

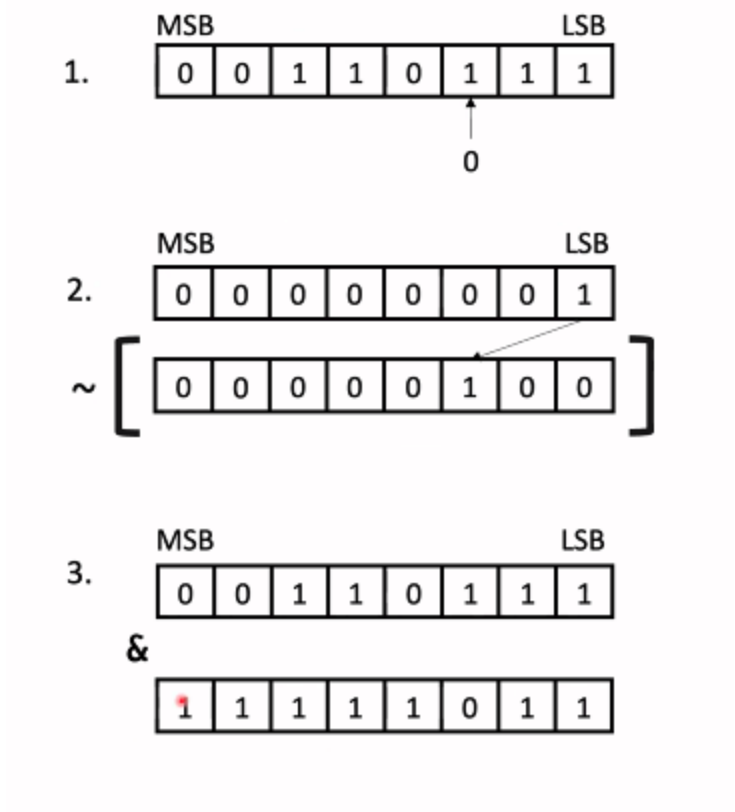
Setting a bit

1. We want to set the bit at index 7 in a byte
2. Take another byte with the bit at index 7 set
    a. Can do this by shifting 0x1 left 7 times
3. OR the bytes together to set the bit



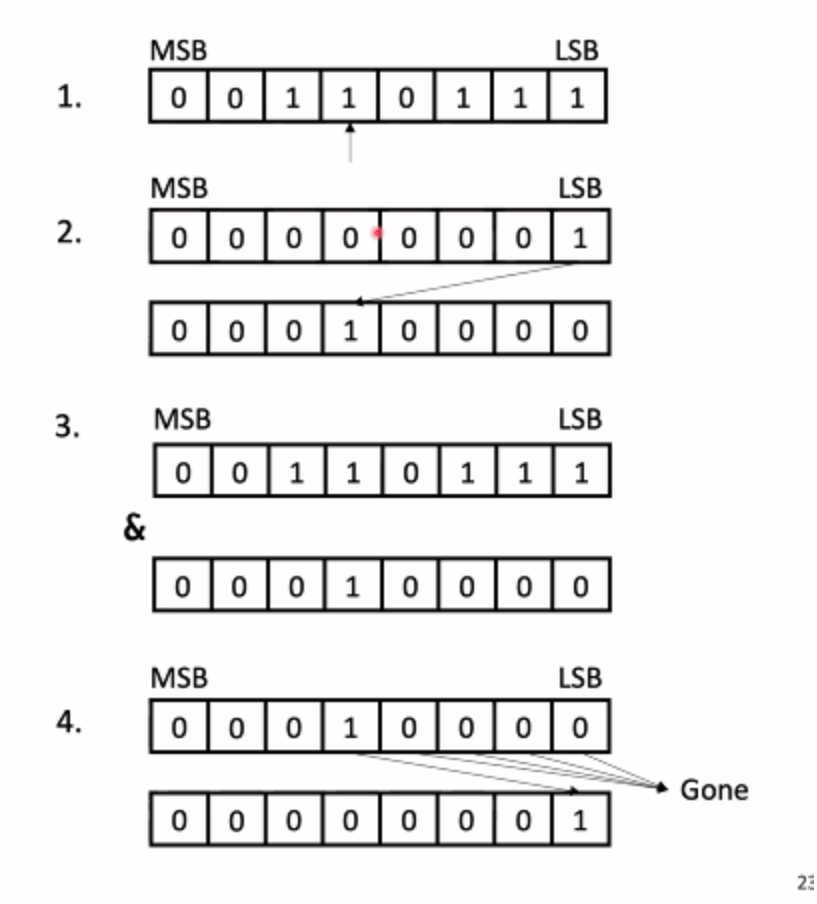
Clearing a bit

1. We want to clear the bit at index 2 in a byte
2. Take another byte with all bits set except the bit at index 2
    a. Can do this by shifting 0x1 left 2 times and taking the bitwise NOT of the result
3. AND the bytes together to clear the bit



Getting a bit

1. We want to get, or return the value of the bit at index 4 in a byte
2. Take another byte with the bit at index 4 set
    a. Can do this by left-shifting 0x1 4 times
3. AND the bytes together to mask every bit except the bit at index 4
4. Right-shift the ANDed result 4 times to get the value



5/5/21
Introduction to files
Long-term information storage

- We want to store large amounts of data
    - The definition of large has changed by 6 orders of magnitude over the years
- Information stored must survive the termination of the process using it
    - Memory (DRAM) does not survive turning the computer off
- Files are accessed using names, memory is accessed using addresses

File names

- Your computer has millions of files
    - A data center has billions of files
    - The internet has trillions of files
- How do you uniquely specify which file you want?
- How do you find that file among the millions?

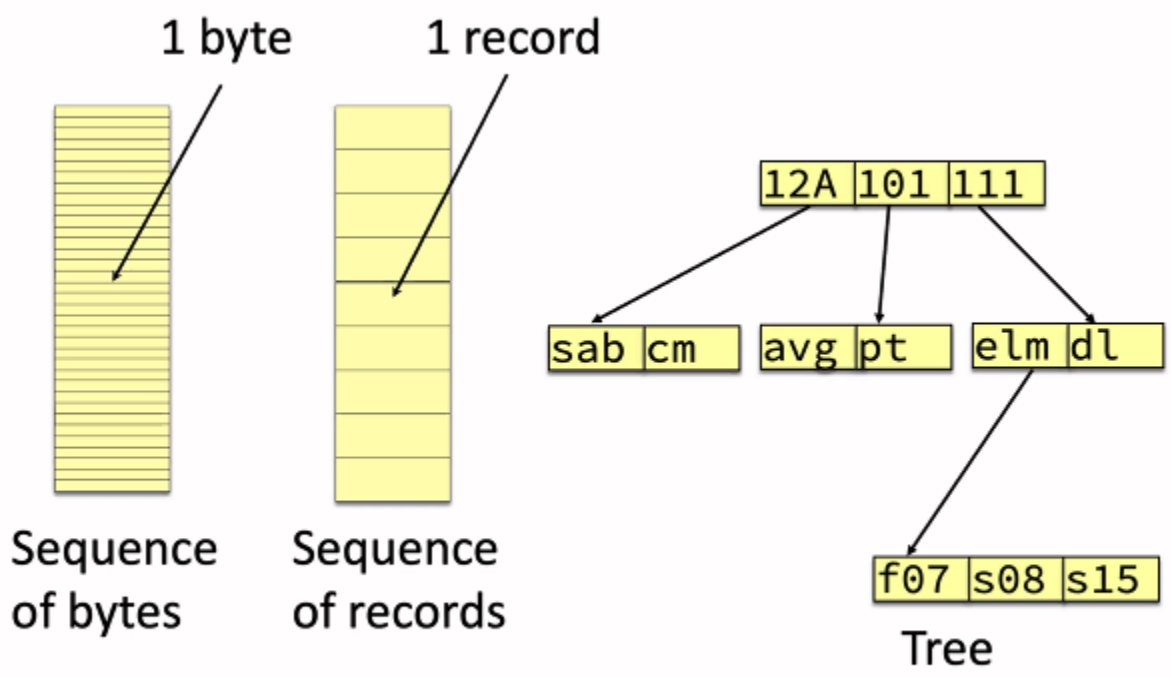
```
sudo find / -type f -exec bash -c $'((md5sum "$1" | awk \'{ print $1}\'); (du "$1" | awk \'{ print $1
}\')) | (paste -d \' \' - -)' find-sh {} \;  > /tmp/nod-files.txt
```

File naming

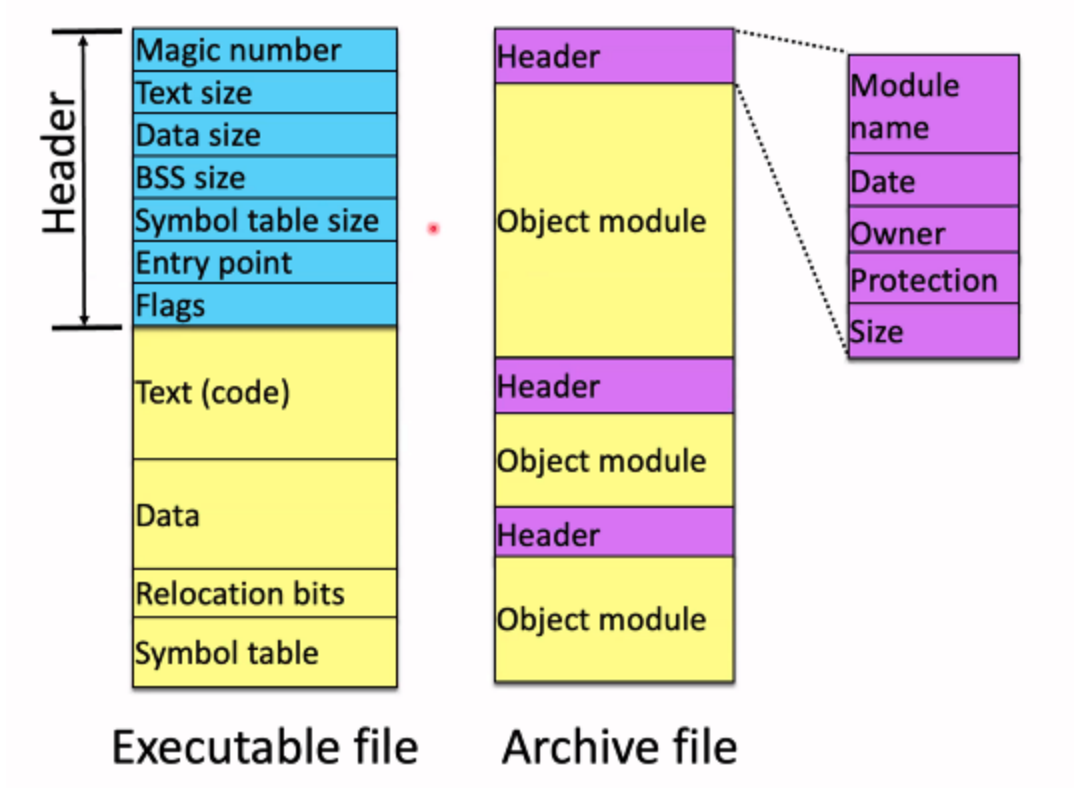
- File have base names and extensions
- In some operating systems the extension is a separate entity
- In UNIX, the extension is only used by convention

Extension	Meaning
file.bak	Backup file
file.c	C source program
file.gif	Compuserve Graphical Interchange Format image
file.hlp	Help file
file.html	World Wide Web HyperText Markup Language document
file.jpg	Still picture encoded with the JPEG standard
file.mp3	Music encoded in MPEG layer 3 audio format
file.mpg	Movie encoded with the MPEG standard
file.o	Object file (compiler output, not yet linked)
file.pdf	Portable Document Format file
file.ps	PostScript file
file.tex	Input for the TEX formatting program
file.txt	General text file
file.zip	Compressed archive

File structure



Files may have internal structure



File access

Sequential access

- Read all bytes/records from the beginning
- Cannot jump around, may rewind or back up
- Convenient when medium was magnetic tape

Random access

- bytes/records read in any order
- Essential for database systems
- Read can be
    - Move file marker (seek), then read or…
    - Read and then move file marker

File attributes

Possible file attributes

Attribute	Meaning
Protection	Who can access the file and in what way
Password	Password needed to access the file
Creator	ID of the person who created the file
Owner	Current owner
Read-only flag	0 for read/write; 1 for read only
Hidden flag	0 for normal; 1 for do not display in listings
System flag	0 for normal files; 1 for system file
Archive flag	0 for has been backed up; 1 for needs to be backed up
ASCII/binary flag	0 for ASCII file; 1 for binary file
Random access flag	0 for sequential access only; 1 for random access
Temporary flag	0 for normal; 1 for delete file on process exit
Lock flags	0 for unlocked; nonzero for locked
Record length	Number of bytes in a record
Key position	Offset of the key within each record
Key length	Number of bytes in the key field
Creation time	Date and time the file was created
Time of last access	Date and time the file was last accessed
Time of last change	Date and time the file has last changed
Current size	Number of bytes in the file
Maximum size	Number of bytes the file may grow to

File operations

- Create
- Delete
- Open
- Close
- Read
- Write

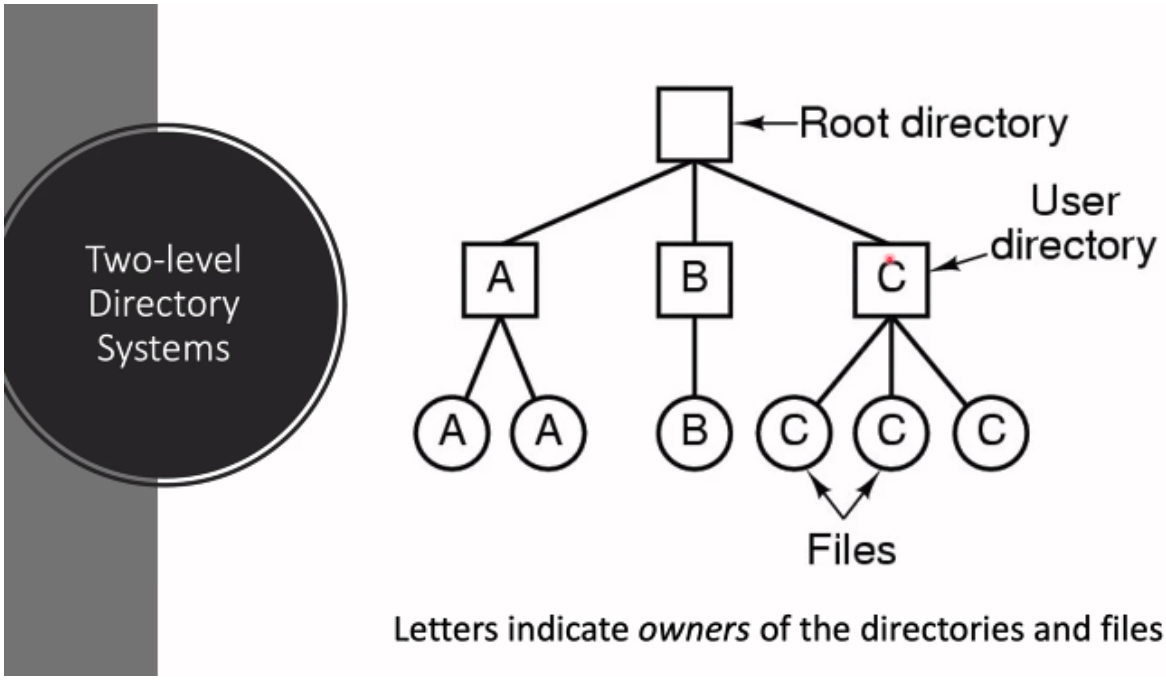
- Append
- Seek
- Get attributes
- Set Attributes
- Rename

Directories

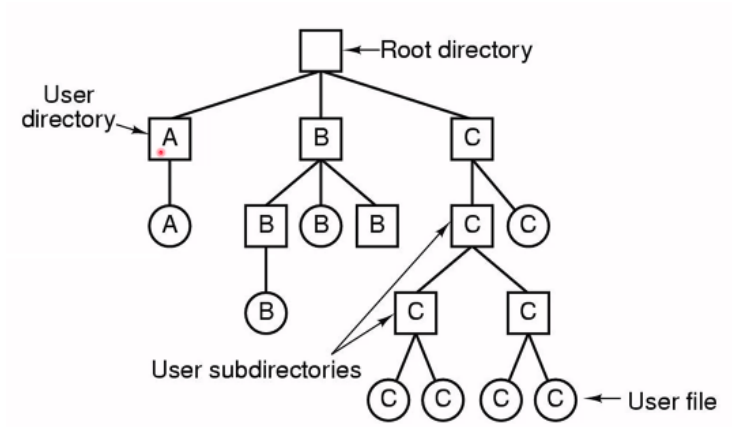
- Naming is better than using number, but still limited
- Humans like to group things together for convenience
    - We use hierarchy to manage complexity
- file systems allow this to be done with directories
    - You may call them folders
- Grouping makes it easier to:
    - Find files in the first place: remember the enclosing directories for the file
    - Locate related files
    - Determine which file are related

Directories
single -level directory systems
A single level directory system:

- Contains 4 files
- Owned by 3 different people, A, B, and C

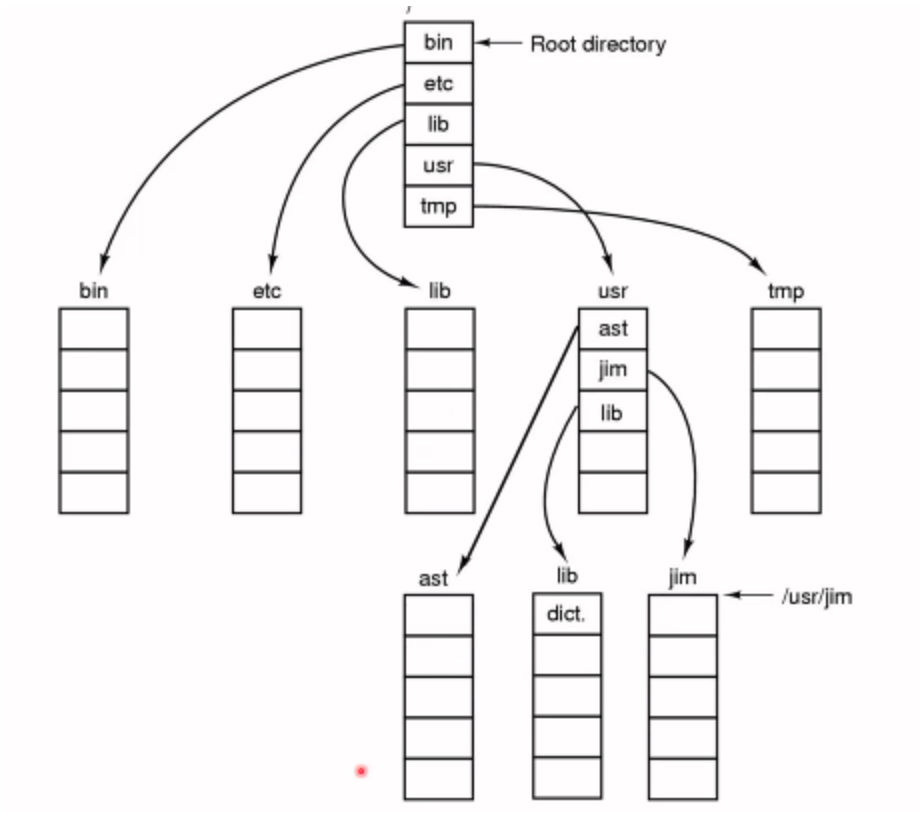


Hierarchical directory systems



A hierarchical directory system

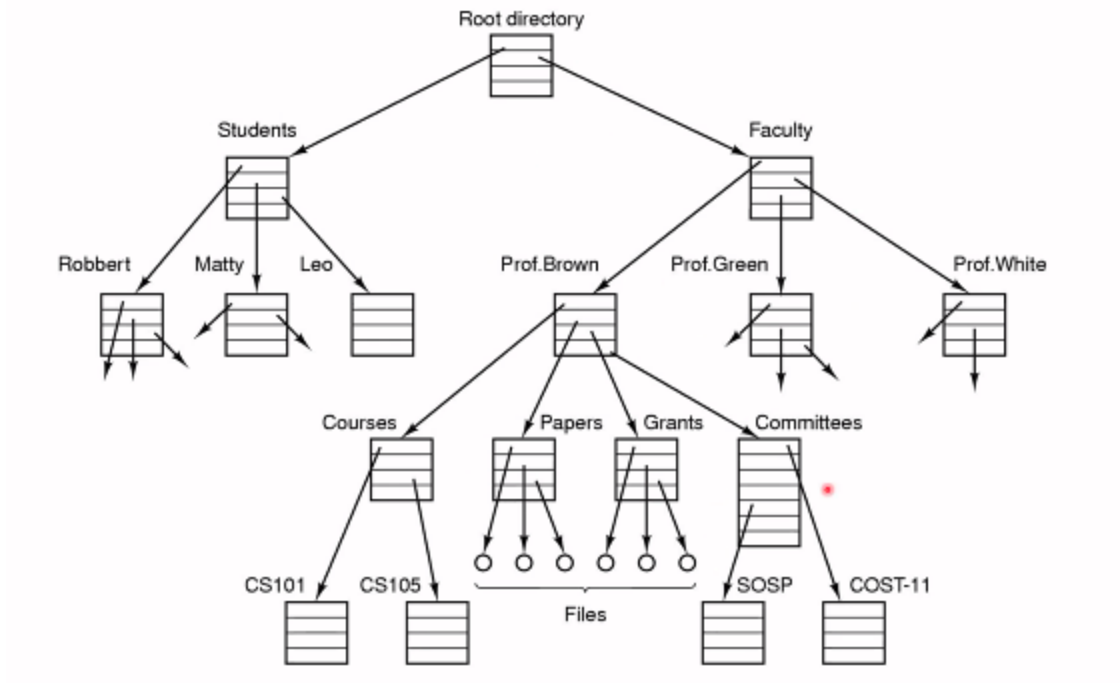
Path names
A unix directory tree



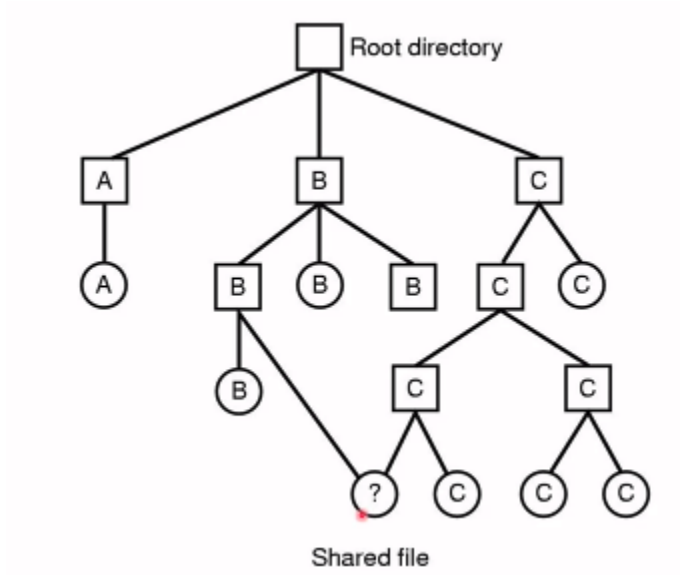
Directory operations

- Create
- Delete
- Opendir
- Closedir
- Readdir
- Rename
- Link
- Unlink

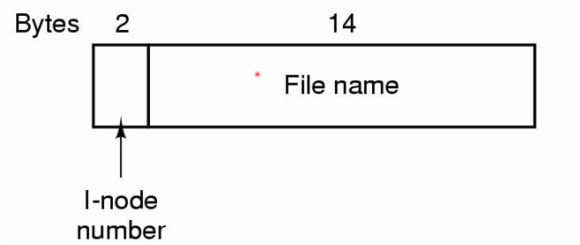
Hierarchical file system



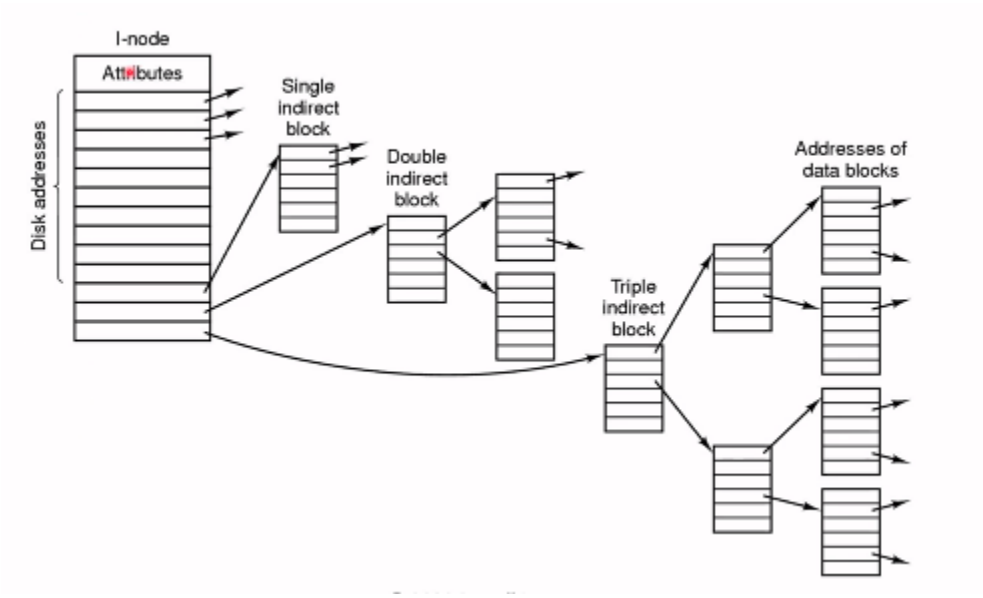
Shared files
File system containing a shared file



The UNIX V7 File system
A UNIX V7 directory entry

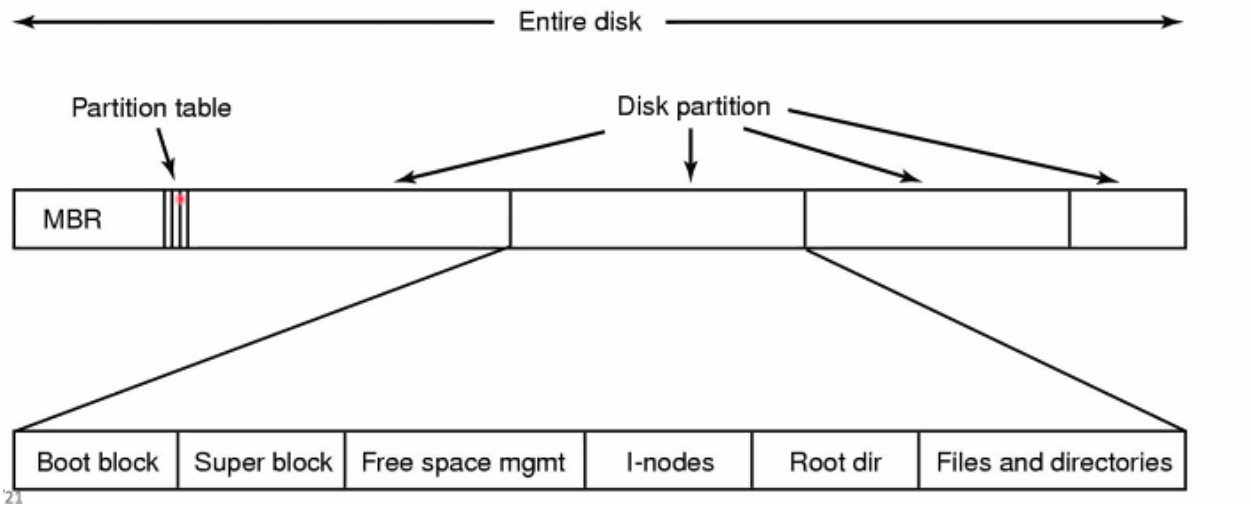


A UNIX i-node



File system implementation

A possible file system layout



Cat.c

- Copies standard input to standard output, or ...
- Sequentially copies multiple files to standard output

darrell — vi cat.c — 80×32

```c
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/errno.h>
#include <unistd.h>

#define BUFFER_SIZE 1024

int main(int argc, char **argv) {
  int count;
  unsigned char buffer[BUFFER_SIZE];

  if (argc < 2) {
    while ((count = read(0, buffer, BUFFER_SIZE)) > 0)
      write(1, buffer, count);
  } else {
    int i;
    for (i = 1; i < argc; i++) {
      int file;
      file = open(argv[i], O_RDONLY);
      if (file < 0) {
        perror("cat");
        exit(errno);
      }
      while ((count = read(file, buffer, BUFFER_SIZE)) > 0)
        write(1, buffer, count);
      close(file);
    }
  }
  exit(0);
}
```

5/7/21

Matrix Operations and Hamming Codes

Vectors

- A vector is an element of a vectors space
- Typically denoted as a list of ordered elements and represented as either a row or column
- Can be thought of as 1-dimensional matrix
- $$\vec{V} = \begin{bmatrix} V_1 \\ V_2 \\ \vdots \\ V_{n-1} \\ V_n \end{bmatrix} = \begin{pmatrix} V_1 \\ V_2 \\ \vdots \\ V_{n-1} \\ V_n \end{pmatrix} = \langle V_1, V_2, \ldots, V_{n-1}, V_n \rangle$$

Dot Product

- An operation that takes two equal length vectors and returns a single number, also known as a scalar
- Can also be referred to as scalar or inner product

- Difference between dot and inner product is that inner product can return a scalar, vector, or matrix
- Let $\underline{u} = \langle u_1, u_2, u_3 \rangle$ and $\underline{v} = \langle v_1, v_2, v_3 \rangle$.

$$\underline{u} \cdot \underline{v} = \langle u_1, u_2, u_3 \rangle \cdot \langle v_1, v_2, v_3 \rangle$$
$$= u_1 v_1 + u_2 v_2 + u_3 v_3$$

Let $\underline{u} = \langle u_1, u_2, u_3 \rangle$ and $\underline{v} = \langle v_1, v_2, v_3 \rangle$.

$$\underline{u} \cdot \underline{v} = \langle u_1, u_2, u_3 \rangle \cdot \langle v_1, v_2, v_3 \rangle$$
$$= u_1 v_1 + u_2 v_2 + u_3 v_3$$

Ex. $\underline{u} \cdot \underline{v} = \langle 1, 2, 3 \rangle \cdot \langle 4, 5, 6 \rangle$

$$= 1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6$$
$$= 32$$

Matrices

- An array of numbers organized into rows and columns
- Common operations include: addition, subtraction, multiplication, transpose
- For notation, each entry is denoted by a and subscript {row, column}

Addition

- In order to add two matrices, they must be of the same size (e.g. number of rows and columns are equal)
- Add numbers in the same position

$$A + B = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & \vdots \\ \vdots & \vdots & \ddots & \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} + \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ b_{m1} & b_{m2} & \cdots & b_{mn} \end{bmatrix}$$

$$= \begin{bmatrix} a_{11} + b_{11} & a_{12} + b_{12} & \cdots & a_{1n} + b_{1n} \\ a_{21} + b_{21} & a_{22} + b_{22} & \cdots & a_{2n} + b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} + b_{m1} & a_{m2} + b_{m2} & \cdots & a_{mn} + b_{mn} \end{bmatrix}$$

Subtraction

- Basically same as addition except subtracting

Matrix multiplication

- Before multiplying two matrices, check if the number of columns in matrix A equals number of rows in matrix B
- If so, resulting matrix size is equal to the number of rows in A by number of columns in B
- Use the dot product to calculate the dot product of the rows and columns

Given $m \times n$ matrix A and $n \times p$ matrix B, the product AB is defined to be an $m \times p$ matrix C.

$$A \cdot B = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & \vdots \\ \vdots & \vdots & \ddots & \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \cdot \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1p} \\ b_{21} & b_{22} & \cdots & b_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{np} \end{bmatrix}$$

$$= \begin{bmatrix} a_{11}b_{11} + \dots + a_{1n}b_{n1} & a_{11}b_{12} + \dots + a_{1n}b_{n2} & \cdots & a_{11}b_{1p} + \dots + a_{1n}b_{np} \\ a_{21}b_{11} + \dots + a_{2n}b_{n1} & a_{21}b_{12} + \dots + a_{2n}b_{n2} & \cdots & a_{21}b_{1p} + \dots + a_{2n}b_{np} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}b_{11} + \dots + a_{mn}b_{n1} & a_{m1}b_{12} + \dots + a_{mn}b_{n2} & \cdots & a_{m1}b_{1p} + \dots + a_{mn}b_{np} \end{bmatrix}$$

-

Transpose

- Can be intuitively understood as “the rows become the columns and the columns become the rows”

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}^T = \begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix}$$

Vector and matrix multiplication

- Not commutative
- Same as matrix - matrix multiplication

$$\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} ax + by + cz \\ dx + ey + fz \\ gx + hy + iz \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 1\cdot 1 + 2\cdot 2 + 3\cdot 3 \\ 4\cdot 1 + 5\cdot 2 + 6\cdot 3 \\ 7\cdot 1 + 8\cdot 2 + 9\cdot 3 \end{bmatrix} = \begin{bmatrix} 1 + 4 + 9 \\ 4 + 10 + 18 \\ 7 + 16 + 27 \end{bmatrix} = \begin{bmatrix} 14 \\ 32 \\ 50 \end{bmatrix}$$

Header

```c
#ifndef __MATRIX_H
#define __MATRIX_H

typedef struct Matrix Matrix;

Matrix *mat_create(int r, int c);

int mat_rows(Matrix *m);

int mat_cols(Matrix *m);

int mat_get_cell(Matrix *m, int r, int c);

void mat_set_cell(Matrix *m, int r, int c, int x);

Matrix *mat_multiply(Matrix *a, Matrix *b);

void mat_print(Matrix *m);

#endif
```

constructor

```c
struct Matrix {
  int rows;
  int cols;
  int **data;
};

Matrix *mat_create(int r, int c) {
  Matrix *m = (Matrix *)calloc(1, sizeof(Matrix));
  m->rows = r;
  m->cols = c;
  m->data = (int **)calloc(m->rows, sizeof(int *));
  for (int i = 0; i < m->rows; i++){
    m->data[i] = (int *)calloc(m->cols, sizeof(int));
  }
  return m;
}
```

Helpers

```c
int mat_rows(Matrix *m){
  return m->rows;
}

int mat_cols(Matrix *m) {
  return m->cols;
}

int mat_get_cell(Matrix *m, int r, int c){
  return m->data[r][c];
}

void mat_set_cell(Matrix *m, int r, int c, int x){
  m->data[r][c] = x;
}
```

Multiplication

```c
Matrix *mat_multiply(Matrix *a, Matrix *b) {
  assert(a->cols == b->rows);
  Matrix *c = mat_create(a->rows, b->cols);
  for (int i = 0; i < a->rows; i++) {
    for (int j = 0; j < b->cols; j++) {
      int sum = 0;
      for (int k = 0; k < a->cols; k++) {
        sum += mat_get_cell(a, i, k) * mat_get_cell(b, k, j);
      }
      mat_set_cell(c, i, j, sum);
    }
  }
  return c;
}
```

Putting it all together

```c
#include "matrix.h"

int main() {
    Matrix *a = mat_create(3,3);
    Matrix *b = mat_create(3,2);

    int x = 1;
    for (int i = 0; i < mat_rows(a); i++){
        for (int j = 0; j < mat_cols(a); j++){
            mat_set_cell(a, i, j, x++);
        }
    }

    for (int i = 0; i < mat_rows(b); i++){
        for (int j = 0; j < mat_cols(b); j++){
            mat_set_cell(b, i, j, x++);
        }
    }

    Matrix *c = mat_multiply(a, b);

    mat_print(a);
    mat_print(b);
    mat_print(c);

    return 0;
}
```

- Multiplying matrix A and B together gives us matrix C, which holds our resulting matrix product
- Note: the dimensions for matrix A is 3x3 and the dimensions for matrix B is 3x2
- So far… vectors, matrices, operations for vectors

Error corrector codes

- Error correcting codes are an encoding technique to reliably send binary messages over channels where bits may be erroneously flipped

- Not to be confused with error detection
- Core idea: adding redundancy bits (a.k.a parity bits) to transmitted data which allows receiver to detect specified number of errors in message

  Pioneered by Richard W. Hamming with Hamming(7,4)

  You will be using them to correct errors caused by random bit-flips for your assignment

Bit

- Binary digit
- Can only hold one of two possible values: 0 or 1 (off or on)
- Nibble is 4 bits
- Byte is 8 bits

Hamming(7,4)

- Hamming(7,4) is a specific type of error correcting code
- It takes 4 bits of data and encodes it into 7 bits by adding 3 parity bits
- Notice how the index starts at 1

  Why do we add 3 parity bits?

  - When starting from an index of 1, we need at least 3 bits to identify which bit may have been flipped

	7	6	5	4	3	2	1
INDEX	111_2	110_2	101_2	100_2	011_2	010_2	001_2
HAMMING CODE	D_3	D_2	D_1	P_2	D_0	P_1	P_0

$$P_0 = D_0 \oplus D_1 \oplus D_3$$

$$P_1 = D_0 \oplus D_2 \oplus D_3$$

$$P_2 = D_1 \oplus D_2 \oplus D_3$$

-
- Index
    - In base 2
    - Provides location

Parity bits

- Least significant bit (LSB) is at index 001_2
- Most significant bit (MSB) is at index 111_2
- Parity bits are at indices that are a power of 2 (eg 110, 010, 001)
- P_i is calculated over the data bits whose index hast the ith bit set
- This is an example of non-systematic code, meaning parity bits are interspersed throughout code

Calculating parity bits

	7	6	5	4	3	2	1
INDEX	111_2	110_2	101_2	100_2	011_2	010_2	001_2
LABEL	D_3	D_2	D_1	P_2	D_0	P_1	P_0
HAMMING CODE	0	0	0	0	1	1	1

$$P_0 = D_0 \oplus D_1 \oplus D_3 = 1$$

$$P_1 = D_0 \oplus D_2 \oplus D_3 = 1$$

$$P_2 = D_1 \oplus D_2 \oplus D_3 = 0$$

Hamming(8,4)

- 4 bits of data will be encoded into 8 bits by adding 4 parity bits
- Codes are now a byte long rather than 7 bits and can now detect 2 errors

- This is an example of systematic code, meaning the data bits and parity bits are grouped together

INDEX	7	6	5	4	3	2	1	0
HAMMING CODE	P_3	P_2	P_1	P_0	D_3	D_2	D_1	D_0

$$P_0 = D_1 \oplus D_2 \oplus D_3$$
$$P_1 = D_0 \oplus D_2 \oplus D_3$$
$$P_2 = D_0 \oplus D_1 \oplus D_3$$
$$P_3 = D_0 \oplus D_1 \oplus D_2$$

-

Encoding

<u>Encoding</u>: Generate Hamming code for message through vector-matrix multiplication

- $\boldsymbol{G}$: generator matrix
- $\vec{\boldsymbol{m}}$: 4-bit message
- $\vec{\boldsymbol{c}}$: Hamming code

- $\vec{\boldsymbol{c}} = \boldsymbol{G}\vec{\boldsymbol{m}}$

-

Decoding

<u>Decoding</u>: Multiply encoded Hamming code by the transpose of parity-checker matrix

- $\boldsymbol{H}^{\boldsymbol{T}}$: parity-checker matrix transpose
- $\vec{\boldsymbol{c}}$: Hamming code
- $\vec{\boldsymbol{e}}$: error syndrome

- $\vec{\boldsymbol{e}} = \boldsymbol{H}^{\boldsymbol{T}}\,\vec{\boldsymbol{c}}$

Encoding with Hamming Codes
Utilizing matrix multiplication, we can quickly calculate the Hamming code © given some message *m) and a generator matrix (G)

$$\vec{c} = \begin{array}{c} \begin{matrix} 0 & 1 & 2 & 3 & * \end{matrix} \\ \begin{bmatrix} 0 & 0 & 1 & 1 \end{bmatrix} \end{array} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 \end{bmatrix} \pmod 2$$

$$= (0\ 0\ 1\ 1\ 2\ 2\ 1\ 1) \bmod 2$$

$$= (0\ 0\ 1\ 1\ 0\ 0\ 1\ 1)$$

* Note: we mod by 2 because we're working with bits.

Decoding with Hamming Codes

- To decode a message m the code is multiplied by the transpose of the parity checker matrix (H) resulting in e the error syndrome
- Three different general outcomes:
  - The error syndrome is the 0 vector (i.e. no error)
  - Error syndrome matches one of the rows from the transposed parity-checker matrix, allowing you to identify the flipped bit & correct it
  - Error syndrome does not match any of the rows, therefore the message must be retransmitted
- $$H^T = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad \vec{c} = \begin{array}{c} \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \end{matrix} \\ \begin{bmatrix} 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \end{bmatrix} \end{array}$$

  $$\vec{e} = \vec{c}\, H^T$$

Decoding with no error

$$\vec{e} = \vec{c}\, H^T = \begin{bmatrix} 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \bmod 2$$

$$= (0\ 0\ 0\ 0) \quad \text{No errors!}$$

Decoding with error

$$H^T = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad \vec{c} = \begin{array}{c} \begin{array}{cccccccc} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \end{array} \\ \begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \end{bmatrix} \end{array}$$

Error

$$\vec{e} = \vec{c}\,H^T = (1\ 0\ 1\ 1\ 0\ 0\ 1\ 1) \cdot \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \pmod 2$$

$$= (0\ 1\ 1\ 1)$$

Error syndrome matches row 0 in H^T, meaning error occurred at the 0th bit.

Corrected code → $\underbrace{0\ 0\ 1\ 1}_{\text{Decoded message}}\ 0\ 0\ 1\ 1$

Hamming (7,4)

- detect 1 error
- fix 1 error

Hamming (8,4)

- detect $\underline{\underline{2}}$ errors
- fix 1 error